

Playwright Automated Testing Tool

Presented by
Saurav Tuladhar

Introduction

Open-source platform developed by Microsoft.

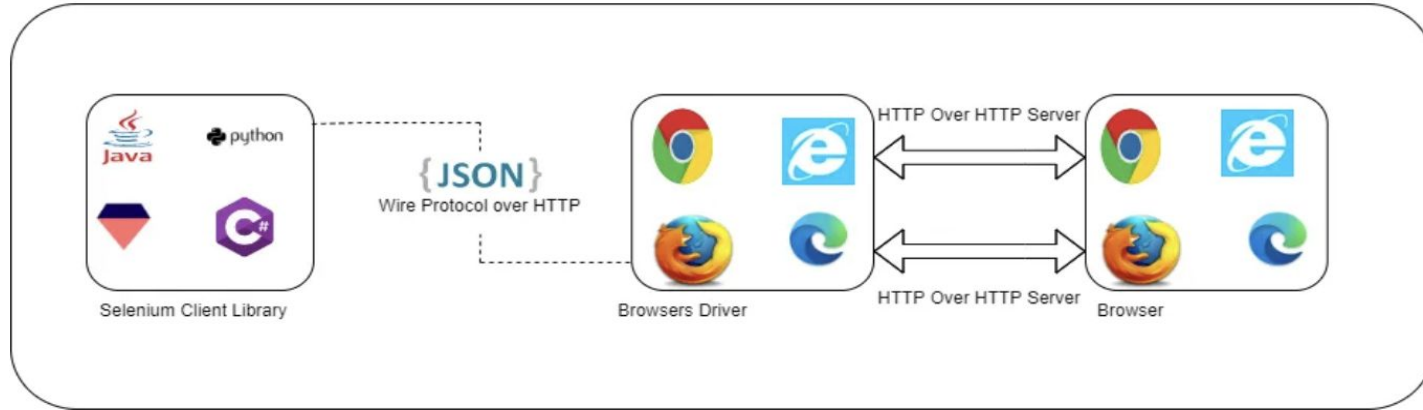
Developed by Microsoft, Playwright is a Node.js library that automates Chromium, Firefox, and WebKit

Playwright Test was created specifically to accommodate the needs of end-to-end testing

Pros and Cons

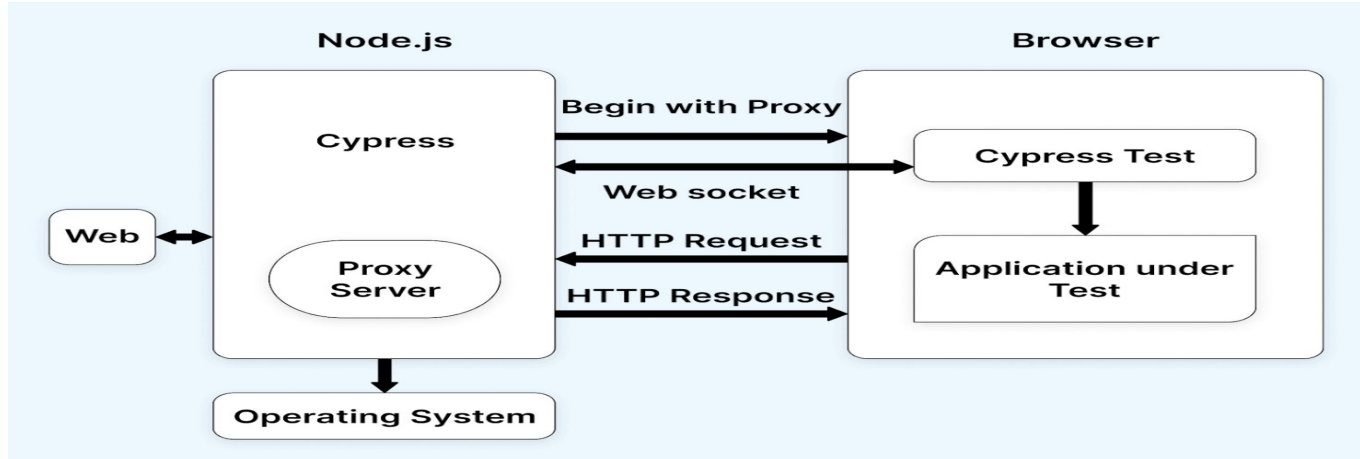
- It supports multi-tab, multi-user, and iframe
- supports different types of testing, such as end-to-end, functional, and API testing.
- Generate an HTML report to view the results of the test run in the browser.
- Auto-waiting of elements
- GUI Interface
- Attachments for test runs
- Does not support Microsoft Edge or earlier IE11.
- Uses a desktop browser instead of a real device to emulate a mobile device
- APIRequest method does not support disabling tracking redirects.
- Confusing documentation
- Smaller community

Architecture Comparison



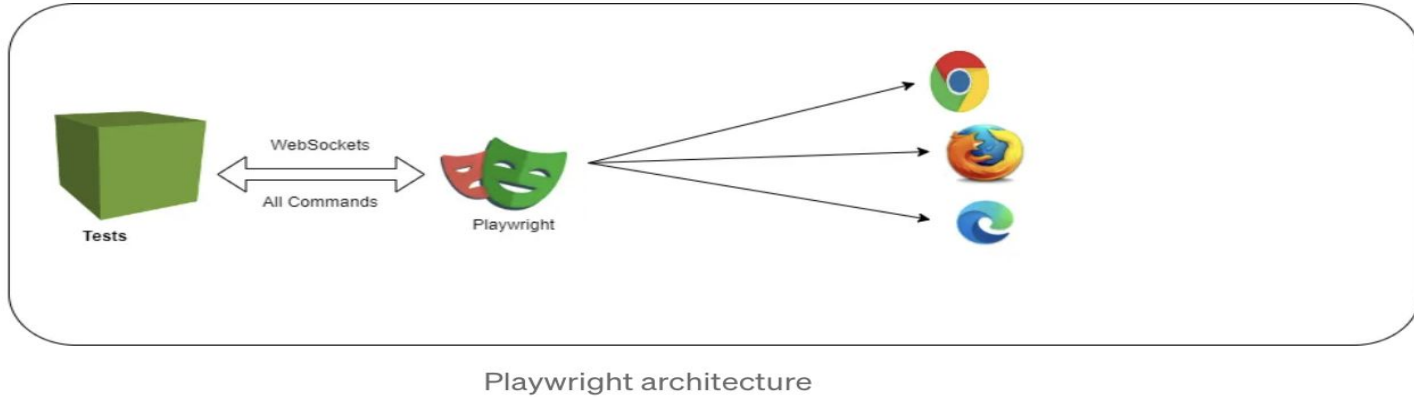
Selenium works on the HTTP connection protocol. It means after you trigger the test, the complete Selenium code written by us (Client) will be converted to JSON format. Generated JSON is sent to the browser driver (Server) through HTTP protocol.

Architecture Comparison



Cypress operates with a Node server, establishing constant communication, synchronization, and task execution between the entities. In Cypress, all commands are executed directly within the browser, eliminating the need for driver binaries and streamlining the testing process.

Architecture Comparison contd..



While Playwright works on Web socket connection protocol, it means once you will trigger the test, the code will be converted into JSON format and will be sent to the server using Web socket protocol.

Annotations

test.beforeAll

test.beforeEach

test.describe

test

test.skip

test.fail

test.only

test.slow

test.afterEach

test.AfterAll

Assertions

Includes test assertions in the form of expect function

To make an assertion, call `expect(value)` and choose a matcher that reflects the expectation

Syntax:

```
await expect(page.getByTestId('status')).toHaveText('Submitted');
```

Assertions contd..

Auto-retrying assertions

Non-retrying assertions

Negating matchers

Soft assertions

Custom expect message

`expect.configure`

`expect.poll`

`expect.toPass`

Installation

Install node.js

`npm init playwright@latest`

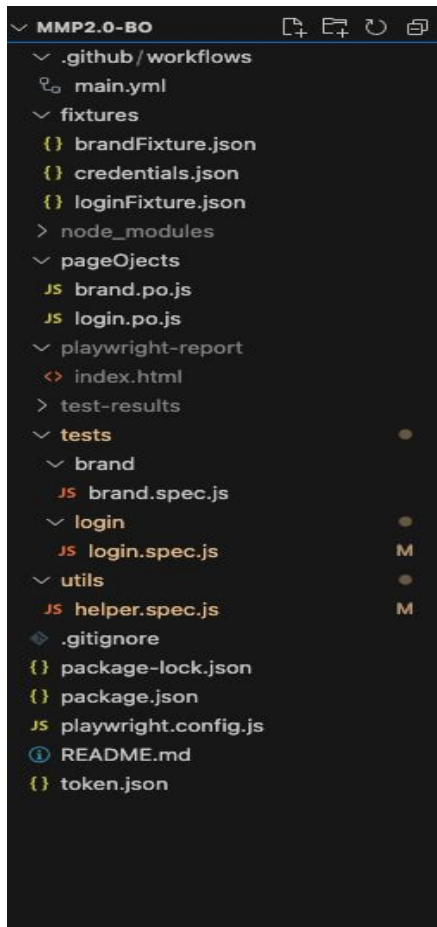
Choose between TypeScript or JavaScript (default is TypeScript)

Name of your Tests folder (default is tests or e2e if you already have a tests folder in your project)

Add a GitHub Actions workflow to easily run tests on CI

Install Playwright browsers (default is true)

Project Structure



- The playwright.config is where you can add configuration for Playwright
- Package.json file contains the dependencies for the project
- The tests folder contains a tests that are to be executed
- Fixtures directory contains the files that are used in automated tests like test data, config files, etc
- Page object contains the web elements locators that are used to trigger actions for certain event
- Playwright-report folder contains the test report that are generated after the tests are completely executed
- Main.yml file is the config file used to run the test in CI

Locating web elements > Xpath

- XPath is a technique that allows you to navigate the structure of a webpage's HTML
- Using XPath helps find elements that are not found by locators such as ID, class, or name
- Types of XPath
 - Absolute XPath
 - Absolute XPath is the direct way to find the element
 - It begins with the single forward slash(/)
 - Select the element from the root node
 - Eg : /html/body/div[1]/div/div[2]/header/div/div[2]/a
 - Relative XPath
 - Path starts from the middle of the HTML DOM structure
 - It starts with the double forward slash (//)
 - It can search the element anywhere at the webpage
 - Eg : //*[@id="block-perfecto-main-menu"]/div[2]/span

Playwright Apis

Playwright Test comes with the built-in request fixture

`Request.post`

`Request.get`

`Request.patch`

`Request.delete`

`Response.status`

`Response.json`

`responseBody.{Key}`

Thank You..!!